

## Practical-1

### Aim:

To implement and analyze the time complexity of various sorting algorithms, namely Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort.

### Algorithm:

Bubble sort  
Algorithm:  
1. Start with the first element  
2. Compare it with the next element  
3. If the first element is greater, swap them  
4. Continue comparing adjacent elements until the end  
5. Repeat the process for all elements  
6. Stop when array is sorted

```
for (i=0; i<n-1; i++) {
    for (j=0; j<n-1-i; j++) {
        if (arr[j] > arr[j+1]) {
            swap(arr[j], arr[j+1]);
        }
    }
}
```

Selection sort  
Algorithm:  
1. Start from the first element of the array  
2. Assume the first element is the minimum element  
3. Compare it with remaining all elements.  
4. Find the smallest element  
5. Swap the smallest element with the first element  
6. Move to the next position  
7. Repeat the same process until the array is sorted  
8. Stop when all elements are in correct positions.

```
for (int i=0; i<n-1; i++) {
    int minIndex = i;
    for (int j=i+1; j<n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    int temp = arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex] = temp;
}
```

Insertion sort  
1. Start from the second element  
2. Store it in Key  
3. Compare Key with the elements before it  
4. Shift larger elements one position to the right  
5. Insert Key in its correct position  
6. Repeat until the array is sorted

```
for (int i=1; i<n; i++) {
    int key = arr[i];
    int j = i-1;
    while (j > 0 && arr[j] > key) {
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
}
```

Merge Sort:

1. Divide the array into two halves
2. Keep dividing until each part has only one element
3. Compare elements of the two parts
4. Merge them in sorted order
5. Repeat merging until the complete array is sorted

```
void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = (l+r)/2;
        mergeSort(a, l, m);
        mergeSort(a, m+1, r);
        merge(a, l, m, r);
    }
}
```

Quick Sort:

1. Choose one element as a pivot
2. Put smaller elements on the left of the pivot
3. Put larger elements on the right of the pivot
4. Apply the same process to the left part
5. Apply the same process to the right part
6. Continue until the array is sorted

```
void QuickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        QuickSort(a, low, p-1);
        QuickSort(a, p+1, high);
    }
}
```

## Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cstdlib>
#include <ctime>
```

```
using namespace std;
```

```
// Helper to print array
```

```
void printArray(const vector<int>& arr) {
    for (size_t i = 0; i < arr.size(); i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
// Bubble Sort
```

```
void bubbleSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

```
    }
  }
}
```

// Selection Sort

```
void selectionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; i++) {
        int min = i;

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min])
                min = j;
        }

        swap(arr[i], arr[min]);
    }
}
```

// Insertion Sort

```
void insertionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}
```

// Merge Sort

```
void merge(vector<int>& arr, int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
```

```
while (i < n1 && j < n2) {  
    if (L[i] <= R[j]) {  
        arr[k] = L[i];  
        i++;  
    }  
    else {  
        arr[k] = R[j];  
        j++;  
    }  
    k++;  
}
```

```
while (i < n1) {  
    arr[k] = L[i];  
    i++;  
    k++;  
}
```

```
while (j < n2) {  
    arr[k] = R[j];  
    j++;  
    k++;  
}  
}
```

```
void mergeSort(vector<int>& arr, int l, int r) {  
    if (l < r) {  
        int m = (l + r) / 2;  
  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
  
        merge(arr, l, m, r);  
    }  
}
```

// Quick Sort

```
int partition(vector<int>& arr, int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
  
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
}
```

```
swap(arr[i + 1], arr[high]);
```

```
        return i + 1;
    }

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    const int n = 20; // smaller size for easy viewing

    vector<int> arr(n), temp;

    srand(time(0));

    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
    }

    cout << "Original Array: ";
    printArray(arr);

    // Bubble Sort
    temp = arr;
    bubbleSort(temp);
    cout << "Bubble Sort: ";
    printArray(temp);

    // Selection Sort
    temp = arr;
    selectionSort(temp);
    cout << "Selection Sort: ";
    printArray(temp);

    // Insertion Sort
    temp = arr;
    insertionSort(temp);
    cout << "Insertion Sort: ";
    printArray(temp);

    // Merge Sort
    temp = arr;
    mergeSort(temp, 0, n - 1);
    cout << "Merge Sort: ";
    printArray(temp);

    // Quick Sort
```

```
temp = arr;
quickSort(temp, 0, n - 1);
cout << "Quick Sort: ";
printArray(temp);

return 0;
}
```

## Output:

```

PS C:\Desktop\dda-practicals> g++ practical1.1.cpp -o practical1.1.exe
PS C:\Desktop\dda-practicals> .\practical1.1.exe
Original Array: 69 55 61 40 79 83 70 17 83 52 44 59 87 14 96 83 54 70 61 69
Bubble Sort: 14 17 40 44 52 54 55 59 61 61 69 69 70 70 79 83 83 87 96
Selection Sort: 14 17 40 44 52 54 55 59 61 61 69 69 70 70 79 83 83 87 96
Insertion Sort: 14 17 40 44 52 54 55 59 61 61 69 69 70 70 79 83 83 87 96
Merge Sort: 14 17 40 44 52 54 55 59 61 61 69 69 70 70 79 83 83 87 96
Quick Sort: 14 17 40 44 52 54 55 59 61 61 69 69 70 70 79 83 83 87 96
PS C:\Desktop\dda-practicals>

```

## Time complexity:

### 1. Bubble sort

Algorithm:

1. Start with the first element
2. Compare it with the next element
3. If the first element is greater, swap them
4. Continue comparing adjacent elements until the end
5. Repeat the process for all elements
6. Stop when array is sorted

```
for (i=0; i<n-1; i++) {  $\rightarrow n-1$ 
    for (j=0; j<n-i-1; j++) {  $\rightarrow n-i-1$ 
        if (arr[j] > arr[j+1]) {
            swap(arr[j], arr[j+1]);
        }
    }
}
```

Bubble Sort:

$$TC(n) = (n-1) + (n-2) + \dots + 1 \Rightarrow (n-1) + (n-2) + \dots + 1$$

$$= \frac{n(n-1)}{2}$$

$$O(n^2) \rightarrow \text{Best Case}$$

### 2. Selection sort:

Algorithm:

1. Start from the first element of the array
2. Assume the first element is the minimum element
3. Compare it with remaining all elements
4. Find the smallest element
5. Swap the smallest element with the first element
6. Move to the next position
7. Repeat the same process until the array is sorted
8. Stop when all elements are in correct positions

```
for (int i=0; i<n-1; i++) {  $\rightarrow n-1$ 
```

```
    int minIndex = i;  $\rightarrow 1$ 
```

```
    for (int j=i+1; j<n; j++) {  $\rightarrow n-i-1$ 
```

```
        if (arr[j] < arr[minIndex]) {
```

```
            minIndex = j;  $\rightarrow 1$ 
```

```
        }
```

```
    }
```

```
    int temp = arr[i];  $\rightarrow 1$ 
```

```
    arr[i] = arr[minIndex];  $\rightarrow 1$ 
```

```
    arr[minIndex] = temp;  $\rightarrow 1$ 
```

```
}
```

Total:

$$n + n-1 + (n-1) + (n-2) + \dots + 1 \Rightarrow \frac{n(n-1)}{2} \Rightarrow O(n^2)$$

### Insertion sort:

1. Start from the second element
2. Store it in key
3. Compare key with the elements before it
4. Shift larger elements one position to the right
5. Insert key in its correct position
6. Repeat until the array is sorted

```
for (int i=1; i<n; i++) {  $\rightarrow n$ 
```

```
    int key = arr[i];  $\rightarrow 1$ 
```

```
    int j = i-1;  $\rightarrow 1$ 
```

```
    while (j>0 && arr[j]>key) {  $\rightarrow i-1$ 
```

```
        arr[j+1] = arr[j];  $\rightarrow 1$ 
```

```
        j--;
```

```
    } arr[j+1] = key;  $\rightarrow 1$ 
```

```
}
```

Total:

$$1 + 2 + 3 + \dots + (n-1)$$

$$\Rightarrow \frac{n(n-1)}{2}$$

$$\Rightarrow O(n^2)$$

### Merge Sort:

1. Divide the array into two halves
2. Keep dividing until each part has only one element
3. Compare elements of the two parts
4. Merge them in sorted order
5. Repeat merging until the complete array is sorted

```
void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = (l+r)/2;
        mergeSort(a, l, m);
        mergeSort(a, m+1, r);
        merge(a, l, m, r);
    }
}
```

Merge Sort recurrence:

$$T(n) = 2T(n/2) + O(n)$$

Compare with:

$$T(n) = AT(n/b) + f(n)$$

$$a=2, \quad b=2, \quad f(n)=n$$

Quick Sort:

1. Choose one element as a pivot
2. Put smaller elements on the left of the pivot
3. Put larger elements on the right of the pivot
4. Apply the same process to the left part
5. Apply the same process to the right part
6. Continue until the array is sorted

```
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p-1);
        quickSort(a, p+1, high);
    }
}
```

$$\text{Partition} = O(n)$$

$$\text{Quick Sort (left)} + \text{Quick Sort (right)}$$

$$T(n) = 2T(n/2) + O(n)$$

By using master method,

$$a=2, \quad b=2, \quad f(n)=n \quad K=1, \quad P=0$$

$$\log_2^2 = 1$$

$$\text{By comparing } K \text{ & } \log_b^a \text{ value } K = \log_b^a$$

So,

$$T(n) = n \log n$$

$$T(n) = n \log n \rightarrow \text{Best/Average case}$$

$$T(n) = O(n^2) \rightarrow \text{Worst case}$$

### **Result:**

The sorting algorithms Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort were successfully implemented and executed. The given elements were sorted in ascending order, and their time complexities were analyzed for different cases. Thus, the implementation and time analysis of the sorting algorithms were successfully completed.